What Four Billion More Parameters Buy — and What They Don't

Qwen2.5-Coder-7B on the spec ladder: a follow-up to "Can a Tiny AI on an 8 GB MacBook Write Real Business Logic?"

Muhammad Junaid — experiments & analysis · **Muhammad Ansar** — original research & problem · 3 July 2026

ABSTRACT

The first report located a sharp boundary for `Qwen2.5-Coder-3B-Instruct-4bit` on a ten-rule pricing spec: with the literal lookup tables in the spec the model reaches 12/12; strip them and it never recovers, failing on the multi-step commission chain. Here we hold everything constant and change one thing — the model, to the 7B of the same family — and follow up with hybrid probes and repair loops at both sizes. Four results. **(1)** The greedy cliff does not move: the 7B fails every de-scaffolded rung at 4/12, exactly where the 3B breaks. **(2)** Scale converts "unreachable" into "rare": best-of-ten sampling reaches 12/12 on all three prose rungs — at a 1–2-in-10 pass-rate, versus 8–10-in-10 for the structured rungs. **(3)** Scale swaps traps rather than removing them: the 7B re-arms the article's original volume-tier bug in a new syntactic disguise, pinning 17 of 20 natural-tier samples at exactly 10/12. **(4)** Paired with a fresh-regeneration white-box repair loop, the 7B converges 12/12 on *every* prose rung within five iterations — including the business-prose rung that resisted every other lever at both sizes. The closing symmetry: the re-dialected v2 spec passes first-try greedy on *both* sizes, while the original spec fails on both — the words, not the weights, remain the lever.

1 Setup: one variable, again

The methodology is inherited unchanged from the first report: eight spec rungs describing the identical ten-rule problem against identical 12-row targets — L1/L2 structured, L3–L5 progressively de-scaffolded prose, L6–L8 controls that keep the lookup tables while re-arming the volume-tier and boolean traps. Frozen harness, frozen instruction wrapper, frozen scorer (pass = 12/12 within 0.5). The single new variable is `--model mlx-community/Qwen2.5-Coder-7B-Instruct-4bit` — same family, same 4-bit quantization, ~4.3 GB against the 3B's ~1.8 GB. Artifacts for each size live in `experiments/sweet-spot/3b/` and `7b/` under identical filenames, and every number below re-scores offline via `prove_ladder.py --dir 7b --tag …`.

2 The greedy cliff does not move

One deterministic temperature-0 attempt per rung, both sizes:

Rung	Spec dialect	3B greedy	7B greedy
L1	Prescriptive pseudocode	12/12	12/12
L2	Declarative spec	9/12	12/12
L3	Business prose	3/12	4/12
L4	Natural rules (tier)	3/12	4/12
L5	Original dialect (boolean)	3/12	4/12
L6	Tables + natural tier	12/12	10/12
L7	Tables + True/False	12/12	12/12
L8	Tables + tier + True/False	12/12	10/12

Table 1 · Greedy, one attempt per rung (row colour = 7B result). The boundary is unchanged: structured specs pass, de-scaffolded specs fail — at either size.

Three details are worth the ink. The 7B repairs the 3B's one greedy wobble (L2, a terse additive volume phrasing, 9/12 → 12/12). On the prose rungs it fails at 4/12 versus the 3B's 3/12 — the extra row comes from cleaner text normalization, not from any progress on the commission chain; inspection shows the same missing-commission signature (only the zero-loading rows pass, plus one). And on two rungs the 7B is *worse* than the 3B (L6, L8: 10/12 vs 12/12) — the subject of §4.

3 Sampling: scale buys reachability, not reliability

Ten attempts per rung at temperature 0.4. The full score distributions make the pattern legible at a glance:

Rung	Spec dialect	7B — ten sampled scores (sorted)	7B pass-rate	3B pass-rate
L1	Prescriptive pseudocode	`12, 12, 12, 12, 12, 12, 12, 12, 12, 12`	10/10	7/10
L2	Declarative spec	`12, 12, 12, 12, 12, 12, 12, 12, 6, 3`	8/10	5/10
L3	Business prose	`12, 12, 5, 5, 5, 4, 4, 4, 4, 0`	2/10	0/10
L4	Natural rules (tier)	`12, 5, 5, 5, 4, 4, 4, 4, 3, 2`	1/10	0/10
L5	Original dialect (boolean)	`12, 12, 5, 5, 4, 4, 4, 4, 4, 0`	2/10	0/10
L6	Tables + natural tier	`12, 10, 10, 10, 10, 10, 10, 10, 10, 10`	1/10	8/10
L7	Tables + True/False	`12, 12, 12, 12, 12, 12, 12, 12, 12, 3`	9/10	1/10
L8	Tables + tier + True/False	`12, 10, 10, 10, 10, 10, 10, 10, 10, 5`	1/10	2/10

Table 2 · The 7B's ten sampled scores per rung (sorted), with pass-rates for both sizes. Green = reliable (≥8/10); red = rare (≤2/10).

Every rung is now *reachable* — the 7B hits 12/12 at least once everywhere, including the three prose rungs the 3B never solved in any experiment. But the distributions show what "reachable" is worth: on L3 the modal outcome is still a 4–5/12 with the commission chain mangled, and the two perfect samples are draws from a long tail. Structure is worth more than scale: the structured rungs run at 8–10/10 while the prose rungs run at 1–2/10. A spec author who writes in the model's dialect gets a better machine than one who buys the bigger model.

4 The trap that came back

The strangest cells in Tables 1–2 are L6 and L8, where the *bigger* model is stuck at exactly 10/12 in 17 of its 20 samples (greedy included) on rungs the 3B passes cleanly. The failing rows are always the two with `volume ≥ 1000`, and the generated code always contains the same construct:

```
volume_rate = 0.05 if volume >= 100 else 0.10 if volume >= 1000 else 0.0
```

A misordered chained ternary: the first condition captures every `volume ≥ 100`, so the `0.10` branch is unreachable and the large-volume rows get half their discount — off by exactly the ratio the scorer shows (e.g. 1.0556 = (1−0.05·…)/(1−0.10·…) on the affected rows). This is the original article's Rule-4 volume-tier bug, resurfacing at 7B in a new syntactic disguise: the 3B collapsed the tier *bounds*; the 7B orders the ternary *conditions* wrong. Meanwhile L7 — which keeps the v2 dialect's additive phrasing ("+0.05 once volume reaches 100, a further +0.05 at 1000") — runs at 9/10 on the 7B and 12/12 greedy on both sizes. **The additive decomposition is robust across model sizes; the natural tier phrasing finds a new way to fail at each size.** Trap-proofing a spec against one model does not certify it for the next.

5 The 7B as knowledge donor: hybrids still fail

If the prose rungs failed for lack of the tables, a bigger model could donate them. In the cross-model hybrid the 7B reads the prose spec and emits the lookup tables as Python literals; the 3B then codes with that extraction appended. The 7B's extractions were **correct in every run** — as were the 3B's own (both sizes rebuild `PRODUCT_RATES` and `CATEGORY_MULTIPLIER` from prose flawlessly, every time). None of it rescues the coder:

Rung	Spec dialect	plain 3B best-of-10	3B extracts, 3B codes	prefilled tables	7B extracts, 3B codes
L3	Business prose	4/12	0/12	4/12	4/12
L4	Natural rules (tier)	3/12	6/12	4/12	1/12
L5	Original dialect (boolean)	6/12	9/12	2/12	0/12

Table 3 · Handing the tables back to the failing rungs (best scores; the 3B is always the coder). Extraction was correct in 100% of runs, both sizes.

With correct tables in hand — its own, or the 7B's — the coder still reads "a 10% surcharge" as `receivable *= 1.10`, never materializes the two loading *amounts*, and the commission has nothing to sum. Knowledge was never the bottleneck; the code-shaped decomposition (bare decimals, named quantities) that accompanies literal tables is what carries the chain.

6 Repair loops: where the 7B breaks through

The white-box repair loop tells the model, per failing row, the *first* pipeline step whose value diverges from a reference ("`commission` = 0.00, should be 3206.25"). Round two rebuilt it with four levers — previous code in the prompt with "edit only the culprit step", a freeze-list of verified steps, three sampled candidates per iteration (keep the best), and a temperature anneal — and the per-iteration score paths tell the whole story:

Variant	L3 score path	L4 score path	L5 score path
anchored (3B) — previous code in prompt	`3 → 3 → 3 → 3 → 3 → 3`	`3 → 3 → 3 → 3 → 3 → 3`	`3 → 3 → 3 → 3 → 3 → 3`
fresh (3B) — regenerate from feedback	`3 → 1 → 0 → 4 → 2 → 0`	`3 → 1 → 2 → 7 → 6 → 9`	`3 → 1 → 1 → 12` ✓
fresh cool-start control (3B)	`3 → 3 → 4 → 0 → 3 → 3 → 0 → 2`	`3 → 3 → 3 → 2 → 0 → 0 → 2 → 0`	`3 → 2 → 2 → 0 → 0 → 2 → 1 → 1`
fresh (7B)	`4 → 4 → 4 → 4 → 12` ✓	`4 → 12` ✓	`4 → 6 → 4 → 10 → 12` ✓

Table 4 · Score after each iteration (row 1 = the greedy attempt; ✓ = converged to 12/12). "Anchored" shows the model its previous code; "fresh" regenerates from feedback alone.

- **Anchoring kills exploration at both sizes.** With its own code in the prompt the model copies it: every candidate, every iteration, an identical score — despite the localization working (culprit named, freeze-list built). The flat paths above are 12 consecutive identical-scoring candidates per rung.
- **Fresh regeneration restores variance** — candidate spreads like `[12, 2, 0]` within a single iteration — and the loop starts climbing. On the 3B that buys back the boolean rung (L5) and lifts L4 to 9/12; L3 stays out of reach, and a cool-start control shows the outcome is a lottery over candidates rather than a schedule effect.
- **On the 7B it closes the board.** All three prose rungs converge — L4 in a single repair iteration (one of its first three candidates scored 12/12), L3 and L5 by iteration five as the anneal reaches 0.3. L3, business prose, is the cell that resisted every other lever at both sizes across both rounds.

The design rule this isolates: give a small model feedback about its code — never the code itself. And the capability rule: the white-box loop is the first lever where scale genuinely pays. Prose in, numerically-correct code out, fully on-device — 7B plus fresh white-box repair.

7 The deliverable spec, at both sizes

The v2 spec — the first report's deliverable, whose dialect the ladder's L1 mirrors — completes the symmetry. Run unchanged on the 7B it passes exactly as it does on the 3B:

Spec	3B	7B
Original ten-rule spec (True/False dialect)	3/12 best	1/12 best
v2 spec — same rules, model's dialect	**12/12 first greedy try**	**12/12 first greedy try**

Table 5 · The original ten-rule spec vs the re-dialected v2, both model sizes (v2 7B run: `solve_simple_v2.py --model …7B…`, saved as `solution_best_v2_7b.py`, re-verified offline).

The original spec fails on both sizes; the v2 dialect passes first-try greedy on both. Nothing about the fix was 3B-specific — it is simply closer to the distribution both coders were trained on.

8 Conclusions

What do four billion extra parameters buy on this task? Not the first attempt: the greedy cliff sits at the same rung, with the same missing-commission signature. Not knowledge: both sizes already rebuild the tables from prose perfectly. What scale buys is *tail probability* — 12/12 exists in the 7B's sample distribution where the 3B's

tail ends at 6 — and, decisively, *repairability*: enough instruction-following headroom that a white-box loop with fresh regeneration converges every prose rung in a handful of iterations. The practical recipe is therefore two-tier: **engineer the spec and the smallest model passes on the first deterministic attempt; keep the prose, and buy the 7B *plus a repair loop* — not the 7B alone.** And one caution travels with it: model-specific trap-proofing does not transfer — each size found its own way to break the natural tier phrasing, and only the explicitly decomposed additive form survived both. The lever, at every size we can fit on this laptop, remains the words.

Models: `mlx-community/Qwen2.5-Coder-3B-Instruct-4bit` and `…-7B-Instruct-4bit` via `mlx-lm 0.29.1` (Python 3.9, Apple Silicon; 16 GB M2 Pro — the 4-bit 7B (~4.3 GB) also fits an 8 GB machine, one model resident at a time). Companion to `slm-tiny-ai-report.pdf` (the round-1 paper, whose §6 this report expands). All artifacts re-score offline: `experiments/sweet-spot/{3b,7b}/outputs/` via `prove_ladder.py`, and `experiments/10-rule-v2/` via `prove_v2.py`. Generated by `docs/build_report_7b.py`.